

Student 10 **KAVI PREETHA G** 192420040RC

10 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

arr = list(map(int, input().split()))
print(selection_sort(arr))
```

INPUT 9 5 1 7

OUTPUT [1, 5, 7, 9]

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

s = input().strip()
if ' ' in s:
    arr = list(map(int, s.split()))
else:
    arr = [int(ch) for ch in s]

print(insertion_sort(arr))
```

INPUT 8 5 7 1 9 3

OUTPUT [1, 3, 5, 7, 8, 9]

Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) ≤ 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

s = input().strip()
if ' ' in s:
    arr = list(map(int, s.split()))
else:
    arr = [int(ch) for ch in s]

print(merge_sort(arr))
```

INPUT

5 2 9 1

OUTPUT

[1, 2, 5, 9]

Q4. Diagonal Matrix

Score: 10.00 TC: 3/3 passed

CODE

```
import json

matrix = json.loads(input().strip())

is_diag = True
for i in range(len(matrix)):
    for j in range(len(matrix[0])):
        if i ≠ j and matrix[i][j] ≠ 0:
            is_diag = False
            break

if is_diag:
    print("Diagonal Matrix")
else:
    print("Not Diagonal Matrix")
```

INPUT

[[1,2],[0,3]]

OUTPUT

Not Diagonal Matrix

Q5. Row and Column Sums

Score: 6.67 TC: 2/3 passed

CODE

```
import sys

raw_input = input().strip()

# Parse the nested list string into a Python 2D list without eval/ast
clean_str = raw_input.replace('[', ' ').replace(']', ' ').strip()
rows = clean_str.split(' , ') if ' , ' in clean_str else raw_input.strip('[]').split(',')

matrix = []
for row in rows:
    row_clean = row.replace('[', '').replace(']', '').strip()
    if row_clean:
        matrix.append([int(x.strip()) for x in row_clean.split(',') if x.strip()])

row_sum = [sum(row) for row in matrix]
col_sum = [sum(col) for col in zip(*matrix)]

print("Row=", row_sum)
print("Col=", col_sum)
```

INPUT

[[1,2],[3,4]]

OUTPUT

Row= [3, 7]
Col= [4, 6]

Q6. Maximum Subarray Sum

Score: 6.67 TC: 2/3 passed

CODE

```
def max_subarray_sum(arr):
    if not arr:
        return 0
    max_so_far = arr[0]
    curr_max = arr[0]

    for i in range(1, len(arr)):
        curr_max = max(arr[i], curr_max + arr[i])
        max_so_far = max(max_so_far, curr_max)

    return max_so_far

arr = list(map(int, input().split()))
print(max_subarray_sum(arr))
```

INPUT

1 2 3 4

OUTPUT

10

Q7. Common Elements Among Three Lists

Score: 10.00 TC: 3/3 passed

CODE

```
parts = input().strip().split(';')

list1 = set(map(int, parts[0].split()))
list2 = set(map(int, parts[1].split()))
list3 = set(map(int, parts[2].split()))

common = sorted(list(list1 & list2 & list3))
print(common)
```

INPUT

1 2;3 4;5 6

OUTPUT

[]

Q8. Matrix Multiplication

Score: 6.67 TC: 2/3 passed

CODE

```
import json

s = input().strip()

a_str = s.split('B=')[0].replace('A=', '').strip()
b_str = s.split('B=')[1].strip()

A = json.loads(a_str)
B = json.loads(b_str)

result = [[sum(A[i][k] * B[k][j] for k in range(len(B))) for j in range(len(B[0]))] for i in range(len(A))]

if len(result) == 1:
    print(result)
else:
    print "[" + str(result[0])
    for row in result[1:-1]:
        print(row)
    print(str(result[-1]) + "]"
```

INPUT

A=[[2]] B=[[3]]

OUTPUT

[[6]]

Q9. Sort Rotate Min Max

Score: 0.00 TC: 0/3 passed

CODE

```
s = input().strip()

parts = s.split(',')
list_part = parts[0].split('=')[1]
k = int(parts[1].split('=')[1])

nums = [int(x) for x in list_part.split()]
sorted_nums = sorted(nums)

n = len(sorted_nums)
k = k % n if n > 0 else 0

# Right rotation matching sample 1 and 3
rotated_nums = sorted_nums[-k:] + sorted_nums[:-k] if k > 0 else sorted_nums[:]

print("Sorted=", sorted_nums)
print("Rotated=", rotated_nums)
print("Min=", min(sorted_nums))
print("Max=", max(sorted_nums))
```

INPUT

List=5 2 8 1 4,k=2

OUTPUT

Sorted= [1, 2, 4, 5, 8]
Rotated= [5, 8, 1, 2, 4]
Min= 1
Max= 8

Q10. Sort Matrix Search

Score: 3.33 TC: 1/3 passed

CODE

```
import json

s = input().strip()

# Extract Matrix and Find values from input format: Matrix=[[ ... ]], Find=X
matrix_str = s.split('Find=')[0].replace('Matrix=', '').strip().rstrip(',')
target = int(s.split('Find=')[1].strip())

matrix = json.loads(matrix_str)
rows = len(matrix)
cols = len(matrix[0])

# Flatten all elements, sort them, and reshape back into matrix dimensions
flat_elements = sorted([val for row in matrix for val in row])
sorted_matrix = [flat_elements[i * cols : (i + 1) * cols] for i in range(rows)]

# Find target position in sorted matrix
position = None
for r in range(rows):
    for c in range(cols):
        if sorted_matrix[r][c] == target:
            position = (r, c)
            break
    if position:
        break

if position:
    print("Sorted=", sorted_matrix)
    print("Position=", position)
else:
    print("Not Found")
```

INPUT

Matrix=[[9,3],[7,1]], Find=7

OUTPUT

Sorted= [[1, 3], [7, 9]]
Position= (1, 0)